

**ПРАВИТЕЛЬСТВО РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
"ВЫСШАЯ ШКОЛА ЭКОНОМИКИ"**

Факультет компьютерных наук
Образовательная программа "Программная инженерия"

СОГЛАСОВАНО

Научный руководитель, научный
сотрудник ФИЦ ИУ РАН

_____ Е. Е. Лимонова
«__» _____ 2026 г.

УТВЕРЖДЕНО

Академический руководитель
образовательной программы
"Программная инженерия", старший
преподаватель Департамента
программной инженерии

_____ Н. А. Павлов
«__» _____ 2026 г.

**БЫСТРАЯ РЕАЛИЗАЦИЯ ВЕЩЕСТВЕННЫХ И КВАНТОВАННЫХ
СВЕРТОЧНЫХ НЕЙРОННЫХ СЕТЕЙ НА МОБИЛЬНЫХ УСТРОЙСТВАХ
APPLE**

Пояснительная записка

ЛИСТ УТВЕРЖДЕНИЯ

RU.17701729.04.06 81 01-1-ЛУ

Исполнители:

Студент группы БПИ241

_____ / А. Д. Ергучев /

«__» _____ 2026 г.

Подп. и дата	
Инв.№ дубл.	
Взам. инв.№	
Подп. и дата	
Инв.№ подл.	

УТВЕРЖДЕН

RU.17701729.04.06 81 01-1-ЛУ

**БЫСТРАЯ РЕАЛИЗАЦИЯ ВЕЩЕСТВЕННЫХ И КВАНТОВАННЫХ
СВЕРТОЧНЫХ НЕЙРОННЫХ СЕТЕЙ НА МОБИЛЬНЫХ УСТРОЙСТВАХ
APPLE**

Пояснительная записка

RU.17701729.04.06 81 01-1

Листов 15

Инов.№ подп	Подп. и дата	Взам. инв.№	Инов.№ дубл.	Подп. и дата

АННОТАЦИЯ

Данный программный документ представляет собой пояснительную записку к программному проекту «Быстрая реализация вещественных и квантованных сверточных нейронных сетей на мобильных устройствах Apple» содержит следующие разделы: «Введение», «Назначение и область применения», «Технические характеристики», «Ожидаемые технико-экономические показатели», приложения.

Раздел «Введение» включает в себя наименование программы и документ, на основании которого ведётся разработка, с указанием организации, утвердившей данный документ.

В разделе «Назначение и область применения» содержатся функциональное и эксплуатационное назначение программы и краткая характеристика области её применения.

В разделе «Назначение разработки» указано функциональное и эксплуатационное назначение создаваемого программного продукта.

В разделе «Технические характеристики» присутствуют следующие подразделы: постановка задачи на разработку программы, описание функционирования программы, описание и обоснование алгоритма работы программы, описание и обоснование выбора метода организации входных и выходных данных, описание работы с базой данных, описание и обоснование выбора состава технических и программных средств.

Раздел «Требования к программным документам» содержит указание на предварительный состав программной документации и специальные требования к ней.

В разделе «Ожидаемые технико-экономические показатели» указана предполагаемая потребность и экономические преимущества разработки по сравнению с отечественными и зарубежными образцами или аналогами.

Настоящий документ разработан в соответствии с требованиями:

1. ГОСТ 19.101-77 [1]: Виды программ и программных документов.
2. ГОСТ 19.102-77 [2]: Стадии разработки.
3. ГОСТ 19.103-77 [3]: Обозначения программ и программных документов.
4. ГОСТ 19.104-78 [4]: Основные надписи.
5. ГОСТ 19.105-78 [5]: Общие требования к программным документам.
6. ГОСТ 19.106-78 [6]: Требования к программным документам, выполненным печатным способом.
7. ГОСТ 19.404-79 [10]: Пояснительная записка. Требования к содержанию и оформлению.

Изменения к данной Пояснительной записке оформляются согласно ГОСТ 19.603-78 [12], ГОСТ 19.604-78 [13].

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.04.06 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

СОДЕРЖАНИЕ

1. ВВЕДЕНИЕ	4
1.1. Наименование программы	4
1.2. Документ(ы), на основании которого(ых) ведется разработка	4
2. НАЗНАЧЕНИЕ РАЗРАБОТКИ	5
2.1. Функциональное назначение	5
2.2. Эксплуатационное назначение	5
2.3. Краткая характеристика области применения программы	5
3. ТЕХНИЧЕСКИЕ ХАРАКТЕРИСТИКИ	7
3.1. Постановка задачи на разработку программы	7
3.2. Описание и обоснование выбора состава технических и программных средств	7
3.2.1. Состав технических и программных средств	7
3.2.2. Обоснование выбора технических и программных средств	7
3.3. Описание и обоснование архитектуры программы	8
3.4. Особенности функционала	8
3.4.1. Модуль управления тензорами	8
3.4.2. Реализация матричного умножения (MatMul)	8
3.4.3. Сверточный слой (Conv2D)	9
3.4.4. Инференс сети LeNet	9
3.4.5. Модуль тестирования и бенчмаркинга	9
3.5. Обоснование выбора метода организации входных и выходных данных	9
4. ОЖИДАЕМЫЕ ТЕХНИКО-ЭКОНОМИЧЕСКИЕ ПОКАЗАТЕЛИ	10
4.1. Ориентировочная экономическая эффективность	10
4.2. Предполагаемая потребность	10
4.3. Экономические преимущества разработки по сравнению с отечественными и зарубежными аналогами	10
ПРИЛОЖЕНИЕ 1. СПИСОК ИСПОЛЬЗУЕМОЙ ЛИТЕРАТУРЫ	12
ПРИЛОЖЕНИЕ 2. ССЫЛКИ НА АНАЛОГИ	13
ПРИЛОЖЕНИЕ 3. ТЕРМИНОЛОГИЯ	14

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.04.06 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

1. ВВЕДЕНИЕ

1.1. Наименование программы

Наименование программы – «Быстрая реализация вещественных и квантованных сверточных нейронных сетей на мобильных устройствах Apple».

Наименование программы на английском языке – «Fast Implementation of Floating-Point and Quantized Convolutional Neural Networks on Apple Mobile Devices».

1.2. Документ(ы), на основании которого(ых) ведется разработка

Разработка ведется на основании учебного плана подготовки бакалавров по направлению 09.03.04 «Программная инженерия» и утвержденной академическим руководителем программы темы курсового проекта.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.04.06 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

2. НАЗНАЧЕНИЕ РАЗРАБОТКИ

2.1. Функциональное назначение

Разрабатываемое программное обеспечение представляет собой библиотеку (API) для высокопроизводительного выполнения прямого прохода (inference) сверточных нейронных сетей на мобильных устройствах с архитектурой Apple Silicon. Основной целью программы является оптимизация вычислений в сверточных слоях и полносвязных слоях за счет использования специализированных аппаратных ресурсов.

Программа должна обеспечивать выполнение следующих функций:

- управление жизненным циклом тензоров (создание, доступ к данным, описание формы и типов данных);
- реализация базовых операций линейной алгебры, включая матричное умножение (GEMM), необходимых для работы сверточных нейросетей;
- обеспечение нескольких путей исполнения вычислений:
 - эталонный путь на CPU для обеспечения корректности и совместимости;
 - ускоренный путь с использованием AMX (Apple Matrix Extensions) и фреймворка Accelerate для повышения производительности;
- поддержка различных типов представлений данных, включая вещественные (floating-point) и квантованные значения;
- реализация стандартных слоев нейронных сетей: сверточный слой (Conv2D), полносвязный слой (Linear), функции активации (ReLU, Softmax) и слои объединения (AvgPool2D);
- преобразование данных из формата свертки в формат матричного умножения с помощью алгоритма im2col;
- диагностика и проверка совместимости размерностей тензоров при выполнении операций с выдачей соответствующих ошибок.

2.2. Эксплуатационное назначение

Программный продукт предназначен для разработчиков мобильных приложений и специалистов по машинному обучению, создающих высоконагруженные системы компьютерного зрения для платформы iOS. Библиотека может быть интегрирована в состав более крупных приложений в качестве бэкенда для выполнения нейросетевых моделей (например, архитектур VGG-уровня) с минимальными задержками и оптимизированным энергопотреблением.

2.3. Краткая характеристика области применения программы

Проект «Быстрая реализация вещественных и квантованных сверточных нейронных сетей на мобильных устройствах Apple» относится к области системного программирования и оптимизации нейросетевых вычислений.

Программа применяется в следующих сценариях:

- встраивание моделей компьютерного зрения в мобильные приложения для анализа изображений в реальном времени;
- сравнительный анализ производительности различных аппаратных путей исполнения (CPU vs AMX) на устройствах Apple Silicon;

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.04.06 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

- реализация инференса квантованных моделей для снижения требований к памяти и ускорения обработки данных без значительной потери точности.

Предметной областью является оптимизация тензорных операций. Основным бизнес-процессом при использовании библиотеки является загрузка весов предобученной модели (в том числе через JSON-сериализацию), подача входного изображения на вход нейросети и получение результата классификации или сегментации.

TODO: вставить график или таблицу сравнения производительности CPU и AMX исполнений.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.04.06 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

3. ТЕХНИЧЕСКИЕ ХАРАКТЕРИСТИКИ

3.1. Постановка задачи на разработку программы

Целью разработки является создание высокопроизводительной программной библиотеки (API) для выполнения прямого прохода (inference) сверточных нейронных сетей на устройствах с архитектурой Apple Silicon.

Основная задача заключается в реализации эффективного тензорного ядра, которое поддерживает как стандартные вещественные вычисления, так и квантованные представления данных. Программа должна обеспечивать возможность переключения между различными путями исполнения (backends), в частности, между эталонным CPU-исполнением и ускоренным исполнением с использованием Apple Matrix Coprocessor (AMX) или фреймворка Accelerate.

Ограничения разработки включают:

- Целевая архитектура: Apple Silicon (ARM64).
- Поддерживаемые операции: матричное умножение (GEMM), свертка 2D (Conv2D), функции активации (ReLU, Softmax), пулинг (AvgPool2D) и операции выравнивания (Flatten).
- Требования к точности: результаты ускоренного исполнения должны совпадать с результатами CPU-эталона в пределах допустимой численной погрешности.

Ожидаемым результатом является C++ библиотека, предоставляющая единый интерфейс для создания тензоров и выполнения операций нейронной сети, оптимизированная под аппаратные возможности мобильных устройств Apple.

3.2. Описание и обоснование выбора состава технических и программных средств

3.2.1. Состав технических и программных средств

Для разработки и тестирования проекта используются следующие средства:

- Оборудование: Устройства на базе Apple Silicon (процессоры серии M или A).
- Операционная система: macOS.
- Язык программирования: C++ (стандарт не ниже C++17 для использования современных возможностей шаблонов и управления памятью).
- Системные библиотеки и фреймворки:
 - Accelerate framework (для доступа к оптимизированным библиотекам BLAS/LAPACK и функциям AMX).
 - Стандартная библиотека C++ (STL).
- Инструменты разработки:
 - Xcode / Clang (компилятор и среда разработки).
 - Python (для реализации скриптов конвертации весов, например, pt2json.py).
 - JSON (формат для сериализации весов нейронной сети).

3.2.2. Обоснование выбора технических и программных средств

Выбор языка C++ обусловлен необходимостью низкоуровневого управления памятью и обеспечения максимальной производительности, что критично для задач инференса нейронных сетей на мобильных устройствах.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.04.06 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

Использование фреймворка Accelerate и прямое обращение к AMX-инструкциям позволяют задействовать специализированные аппаратные блоки Apple Silicon, которые на порядки превосходят обычные векторные инструкции NEON при выполнении матричного умножения.

Формат JSON был выбран для хранения весов модели (`mnist_lenet.json`) из-за его прозрачности и простоты парсинга, что облегчает перенос обученных моделей из PyTorch в разрабатываемую библиотеку с помощью вспомогательных Python-скриптов.

Выбор архитектуры Apple Silicon в качестве целевой позволяет исследовать специфические возможности современных SoC, такие как объединение памяти (Unified Memory Architecture), что сокращает накладные расходы при передаче данных между различными модулями вычислений.

3.3. Описание и обоснование архитектуры программы

Архитектура программы построена по модульному принципу с четким разделением между уровнем представления данных, уровнем операторов и уровнем аппаратного исполнения (backends).

Основные компоненты архитектуры:

1. Слой представления данных (Core Tensor): Состоит из классов `Tensor`, `TensorDesc` и `TensorView`. `TensorDesc` описывает форму (`shape`), тип данных (`dtype`) и режим вычислений, а `Tensor` управляет физическим выделением памяти.
2. Слой базовых операций (Ops): Содержит реализацию элементарных операций (например, `elementwise.hpp`, `matmul.hpp`). Этот слой абстрагирует конкретную реализацию вычислений.
3. Слой бэкендов (Backends): Реализует конкретные пути исполнения.
 - CPU-backend: Эталонная реализация для проверки корректности.
 - AMX/Accelerate-backend: Высокопроизводительная реализация матричных операций.
4. Слой нейронных сетей (NN): Высокоуровневые модули (`Layer`, `Conv2D`, `Linear`, `LeNet`), которые объединяют базовые операции в полноценные слои и архитектуры сетей.
5. Система сериализации: Модуль `serialization.hpp`, обеспечивающий загрузку весов из JSON-файлов в тензоры.

Такая структура позволяет добавлять новые методы оптимизации (например, NEON-оптимизации для GEMM) без изменения логики работы верхних слоев нейронной сети.

3.4. Особенности функционала

3.4.1. Модуль управления тензорами

Назначение: Обеспечение единого интерфейса для работы с многомерными массивами данных. Входные данные: Описание формы (размерности), тип данных (`float`, квантованные типы). Действия: Выделение памяти, проверка совместимости размерностей при операциях, итерация по данным через `TensorIterator`. Результат: Созданный объект тензора, готовый к вычислениям.

3.4.2. Реализация матричного умножения (MatMul)

Назначение: Выполнение основной вычислительной нагрузки сверточных и линейных слоев. Входные данные: Два тензора (матрицы) совместимых размерностей. Действия: Система выбирает путь исполнения (AMX или CPU). При использовании AMX данные передаются в специ-

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.04.06 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

ализированные регистры сопроцессора. При отсутствии поддержки или ошибке происходит откат (fallback) к CPU-реализации. Результат: Тензор с результатом произведения матриц.

3.4.3. Сверточный слой (Conv2D)

Назначение: Извлечение признаков из входного изображения. Входные данные: Входной тензор (image), тензор весов (filters). Действия: Преобразование свертки в матричное умножение с помощью алгоритма im2col, запуск MatMul, применение смещения (bias) и функции активации. Результат: Тензор с картами признаков.

3.4.4. Инференс сети LeNet

Назначение: Полный цикл распознавания образа (например, цифр MNIST). Входные данные: Изображение в формате тензора, файл с весами mnist_lenet.json. Действия: Последовательный проход данных через слои Conv2D -> ReLU -> AvgPool2D -> Flatten -> Linear -> Softmax. Результат: Вектор вероятностей классов.

3.4.5. Модуль тестирования и бенчмаркинга

Назначение: Верификация точности и замер производительности. Входные данные: Тестовые изображения (test1_9.jpg и др.), эталонные результаты. Действия: Сравнение выходных тензоров AMX-пути и CPU-пути с вычислением численной погрешности. Замер времени выполнения операций. Результат: Отчет о корректности и графики производительности.

TODO: вставить скрин графиков сравнения производительности CPU vs AMX

3.5. Обоснование выбора метода организации входных и выходных данных

Входные данные (веса моделей) организованы в виде JSON-файлов, что обеспечивает переносимость между средой обучения (PyTorch) и средой исполнения. Процесс загрузки реализован через модуль сериализации, который распределяет данные по соответствующим тензорам каждого слоя.

Внутреннее представление данных базируется на плоских массивах в памяти с использованием TensorDesc для вычисления смещений (offsets) при доступе к элементам многомерных тензоров. Это позволяет минимизировать количество аллокаций и использовать эффективные методы обхода памяти.

Обмен данными между компонентами осуществляется по meiou ссылок на объекты Tensor или через TensorView, что предотвращает избыточное копирование больших объемов данных при передаче между слоями нейронной сети.

Выходные данные программы представлены в виде тензоров с результатами активаций последнего слоя (например, Softmax), которые затем интерпретируются как предсказания классов нейронной сети.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.04.06 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

4. ОЖИДАЕМЫЕ ТЕХНИКО-ЭКОНОМИЧЕСКИЕ ПОКАЗАТЕЛИ

4.1. Ориентировочная экономическая эффективность

Экономическая эффективность предлагаемого решения заключается в оптимизации использования вычислительных ресурсов мобильных устройств Apple Silicon за счет прямого использования специализированных инструкций AMX (Apple Matrix Extensions) и библиотеки Accelerate.

Ожидаемый эффект от внедрения включает:

- Снижение энергопотребления устройства при выполнении задач инференса нейронных сетей, что напрямую увеличивает время автономной работы мобильного устройства.
- Повышение пропускной способности системы (throughput) при обработке потоков данных компьютерного зрения (VGG-уровни), что позволяет выполнять более сложные задачи анализа в реальном времени без задержек.
- Снижение требований к аппаратной части за счет поддержки квантованных представлений данных, что позволяет запускать более тяжелые модели на устройствах с ограниченным объемом оперативной памяти.
- Сокращение времени разработки при интеграции в конечные продукты благодаря наличию единого C++ API, который абстрагирует сложность переключения между CPU-эталоном и ускоренным AMX-путем исполнения.

4.2. Предполагаемая потребность

Целевой аудиторией разработки являются разработчики прикладного программного обеспечения для iOS/macOS, создающие высоконагруженные приложения в области компьютерного зрения, дополненной реальности (AR) и мобильной аналитики.

Разработка востребована в следующих ситуациях использования:

- Реализация локального инференса сверточных нейронных сетей (CNN) без передачи данных на удаленный сервер, что критично для обеспечения приватности пользователей и работы в режиме оффлайн.
- Создание приложений, требующих минимального времени отклика (low latency) при обработке изображений, где стандартные высокоуровневые библиотеки могут иметь избыточные накладные расходы.
- Оптимизация моделей под конкретную архитектуру Apple Silicon для достижения максимального аппаратного ускорения матричных операций.

Необходимость в данном проекте обусловлена отсутствием открытых и легко интегрируемых C++ библиотек, предоставляющих прозрачный доступ к AMX-инструкциям с одновременной поддержкой как вещественной, так и квантованной арифметики.

4.3. Экономические преимущества разработки по сравнению с отечественными и зарубежными аналогами

В сравнении с существующими решениями (такими как CoreML, TensorFlow Lite или ONNX Runtime), данный проект ориентирован на предоставление низкоуровневого контроля над исполнением матричных операций и минимальный размер бинарного файла.

Ключевые преимущества:

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.04.06 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

- **Специализация:** В отличие от универсальных фреймворков, библиотека сфокусирована на VGG-подобных структурах и максимально эффективном использовании AMX.
- **Гибкость типов:** Поддержка различных разрядностей квантования позволяет более точно балансировать между точностью вычислений и скоростью работы.
- **Проверяемость:** Наличие встроенного CPU-эталона позволяет проводить верификацию корректности вычислений непосредственно в процессе разработки.
- **Отсутствие зависимостей:** Минимизация внешних зависимостей упрощает процесс сборки и снижает стоимость поддержки продукта.

Характеристика	CoreML	TFLite	ONNX Runtime	Our Project
Низкоуровневый доступ к AMX	-	-	-	+
Минимальный размер runtime	-	+	-	+
Поддержка квантования	+	+	+	+
Простой C++ API без тяжелых зависимостей	-	-	-	+
Встроенный CPU-эталон для тестов	-	-	-	+
Итого	1	2	1	5

Таблица 1. Сравнение функциональных и эксплуатационных характеристик с аналогами

TODO: вставить графики сравнения производительности (CPU vs AMX) по результатам бенчмарков lenet_benchmark.cpp

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.04.06 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

ПРИЛОЖЕНИЕ 1. СПИСОК ИСПОЛЬЗУЕМОЙ ЛИТЕРАТУРЫ

1. ГОСТ 19.101-77: Виды программ и программных документов. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
2. ГОСТ 19.102-77: Стадии разработки. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
3. ГОСТ 19.103-77: Обозначения программ и программных документов. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
4. ГОСТ 19.104-78: Основные надписи. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
5. ГОСТ 19.105-78: Общие требования к программным документам. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
6. ГОСТ 19.106-78: Требования к программным документам, выполненным печатным способом. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
7. ГОСТ 19.201-78: Техническое задание. Требования к содержанию и оформлению. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
8. ГОСТ 19.301-79: Программа и методика испытаний. Требования к содержанию и оформлению. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
9. ГОСТ 19.401-78: Текст программы. Требования к содержанию и оформлению. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
10. ГОСТ 19.404-79: Пояснительная записка. Требования к содержанию и оформлению. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
11. ГОСТ 19.505-79: Руководство оператора. Требования к содержанию и оформлению. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
12. ГОСТ 19.603-78: Общие правила внесения изменений. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
13. ГОСТ 19.604-78: Правила внесения изменений в программные документы, выполненные печатным способом. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.04.06 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

ПРИЛОЖЕНИЕ 2. ССЫЛКИ НА АНАЛОГИ

TODO: Таблица со списком ссылок на аналоги. ПРИМЕР:

Приложение	Ссылка
Яндекс Музыка	https://music.yandex.ru/
Spotify	https://www.spotify.com/
ВК Музыка	https://music.vk.com/
YouTube Music	https://music.youtube.com/
Apple Music	https://music.apple.com/
Black Candy	https://github.com/blackcandy-org/blackcandy
Navidrome	https://www.navidrome.org/
Ampache	https://ampache.org/
mStream	https://mstream.io/

Дата обращения: 05.03.25.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.04.06 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

ПРИЛОЖЕНИЕ 3. ТЕРМИНОЛОГИЯ

TODO: написать список используемых терминов и их значения. ПРИМЕР:

Термин	Определение
Фреймворк	Структурированная платформа, включающая набор инструментов, библиотек и правил, разработанных для упрощения создания и управления программным обеспечением.
Стриминг	Технология непрерывной передачи аудио- или видеоконтента через сеть, при которой данные поступают и воспроизводятся в режиме реального времени, без необходимости предварительной загрузки полного файла.
Селфхостинг	Метод организации хостинга, при котором пользователь или организация самостоятельно управляют серверной частью и хранением данных, без использования сторонних облачных сервисов.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.04.06 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

ЛИСТ РЕГИСТРАЦИИ ИЗМЕНЕНИЙ

[illegible]